

Exercise 5: Variational Auto-Encoders

*Dan Rosenbaum***Due: 8 June 2026**

In this exercise we will use the following skeleton notebook:

<https://colab.research.google.com/drive/1l2vgJCpgHJFJ4HMkw6Ytsrm8EZnel9mo?usp=sharing>

The goal of this exercise is to implement a variational autoencoder (VAE) model.

In order to implement a VAE you will need to implement both an encoder network and a decoder network.

- The **encoder** should consist of several layers of convolutions (you can use stride=2 in two of the layers to gradually decrease the dimension of the representation), followed by several layers of fully connected linear layers.

The encoder is used to define a posterior probability over the latents $q(z|x)$. The posterior should be implemented as a Gaussian with different mean and variance for each dimension of the latent z . The way this can be implemented is by mapping the output of the decoder (using learned linear mappings) to two vectors with the same dimension as the latents. The first vector is used as the means, and the second vector is used as the log of the std (exponentiate it before you pass it to the distribution).

- The **decoder** should consist of several layers of fully connected linear layers followed by several layers of transposed convolutions [link] such that the output has the same shape as the input images. Add a sigmoid activation at the end to obtain output which is in the range $[0,1]$.

This output will be used as the mean of the reconstruction distribution $p(x|z)$. You can use a constant variance for this distribution, which will be a hyperparameter that you can tune. A good value to start from is $\text{var}(x|z) = 0.1^2$. This means that the reconstruction term in the ELBO, $-\log p(x|z)$ simplifies to $\frac{1}{2 \cdot 0.1^2} \|\hat{x} - x\|^2$, where $\hat{x}$ is the output of the decoder and x is the ground truth image.

- In addition to implementing an encoder and a decoder, you will need to define a prior distribution. You should use a standard isotropic Gaussian over the latent vector, without learned parameters. Note that the prior, encoder and decoder are used in different ways for computing the ELBO and for sampling.

You will use the same training process and model structure as in the previous exercise (pixelCNN). This means you will need to implement two main functions:

1. **calc_elbo:** As computing the likelihood is intractable, you should implement a function that calculates the ELBO. This should replace 'calc_likelihood' and be used for training, evaluation and testing. As we defined in class, the ELBO consists of two terms: the reconstruction log likelihood, $\log p(x|z)$, and the KL divergence, $D_{KL}(q(z|x)||p(z))$. Note that in the previous exercise the functions returned the negative likelihood so you can also return the negative ELBO here.
2. **sample:** This function is used to generate samples. The parameters of this function should only be the number of requested samples. The output of the function should be the output of the decoder, i.e. the means of $p(x|z)$. This is not a full sampling process because the constant variance per pixel is not added but it is better for most purposes as it does not add per-pixel noise.

Recall that in order to compute the ELBO for training, you should sample from the posterior in a way that is differentiable. The way this is implemented in pytorch is using the reparameterization trick we discussed in class, using the function 'rsample'. See the documentation.

Continuous representation: In order to consider the pixel values as continuous, you should cast them to a float type, add uniform noise in the range of $[0, 1]$, and then scale them from $[0, 256]$ to $[0, 1]$. This should be done at the preprocessing step, as a transform operation.

Assignments

1. Implement a VAE as described above and train it on the MNIST data.

Train this for at least 20 epochs. Report the ELBO for the test set, curves showing the ELBO during training (for both training and validation), and curves showing the reconstruction likelihood and KL divergence during training. After training, generate a few samples and show the results. You should be able to train models where most of the samples look like valid digits.

2. In this assignment you will generate latent traversals, showing the representation power of VAEs. Use the model that you trained, and chose two images of different digits from the dataset. For each image, compute a sample from the posterior $q(z|x)$. This will be the 'representation' of the image. Now given the two latent representations z_1, z_2 , compute a linear path between them with a set of 10 points along the way. For each of those 10 points, z^i , compute the reconstruction sample $p(x|z^i)$ (you can use the mean of this distribution like before), and show the resulting images in order. You should see how the first digit transforms into the second digit. Do the same thing in pixel space, by interpolating directly between the pixels of the images. Repeat this for 3 different pairs of images, and submit a figure that contains a total of 6 rows of traversals consisting of 3 examples where each has one row for the latent-space and one row for the pixel space.

3. In this question you will use the trained VAE to perform image inpainting. The task: given the top half of an MNIST image x_{obs} , predict a plausible bottom half. Let $m \in \{0, 1\}^{784}$ be a mask with $m = 1$ on the observed (top) pixels and $m = 0$ on the missing (bottom) pixels.

You will implement three different inpainting methods, all of which use only the *frozen* trained VAE.

- (a) **Iterative encode–decode with clamping.** Initialise the missing pixels with uniform random values in $[0, 1]$ to form a full image x_0 . Then repeatedly run the image through the encoder and decoder, clamping the observed pixels back to x_{obs} after each pass:

$$z_t \sim q(z | x_t) \quad (\text{or use the posterior mean}) \quad x'_t = \mu_\theta(z_t) \quad x_{t+1} = m \odot x_{\text{obs}} + (1 - m) \odot x'_t.$$

Run for about 30 iterations and show snapshots of the image at several iterations.

- (b) **MAP estimate of z via gradient descent (decoder frozen, latent variable).** Find the latent vector z that best explains the observed half under the prior. Initialise $z \sim \mathcal{N}(0, I)$ and minimise

$$L(z) = \underbrace{\frac{1}{2\sigma^2} \|(\mu_\theta(z) - x_{\text{obs}}) \odot m\|^2}_{-\log p(x_{\text{obs}}|z)} + \underbrace{\frac{1}{2} \|z\|^2}_{-\log p(z)}$$

with respect to z using Adam (and the same σ as the decoder). After convergence, decode $\mu_\theta(z^*)$ and fill in the missing pixels.

- (c) **Maximum likelihood on x via gradient descent (VAE frozen, image pixels).** Now keep the entire VAE fixed and instead optimise the missing pixels directly to maximise the model's likelihood of

the full image. Parameterise the free pixels via a sigmoid so they stay in $[0, 1]$: $x_{\text{free}} = \sigma(y)$, and form the full image as $x(y) = m \odot x_{\text{obs}} + (1 - m) \odot x_{\text{free}}$. Minimise the negative ELBO with respect to y :

$$L(y) = -\text{ELBO}(x(y)) = -\log p(x(y) | z) + D_{KL}(q(z | x(y)) \| p(z)),$$

where z is sampled from $q(z | x(y))$ at each step (reparameterised, so the gradient flows back into y).

Run all three methods on the same 6 test images. For each method, produce a figure with rows = examples and columns = [observed half | ground truth | result]. Also include the loss curves for (b) and (c). Briefly compare the three methods and explain the qualitative differences you observe. Think about what role the encoder, the prior, and the reconstruction term each play in (a), (b) and (c), and why methods that are mathematically reasonable can still fail in practice (e.g. because of local minima or because the optimum of the surrogate objective is not the optimum of the original task).

You should submit a colab notebook that contains the text of the answers, figures and code for all the above questions.